DSA PROJECT

# EventFlow

## Large-Scale Event Management Platform

A data-driven platform for event discovery, intelligent search, scheduling and resource allocation.


PROJECT TEAM

**P. Saanvi • S. Vishalini • A. SivaTejaswi**


Professional project overview | DSA modules M1–M6

# Project at a glance

### Discover

Search events, explore descriptions and find relevant activities.

### Coordinate

Organize venues, schedules, staff and volunteer assignments.

### Understand

Use algorithms and analytics to support decisions.

THE PROBLEM

**Large events generate scattered records, repeated descriptions, booking searches and resource conflicts.**

THE SOLUTION

**A unified event platform that combines operational workflows with demonstrable Data Structures and Algorithms.**

# Problem statement & objectives

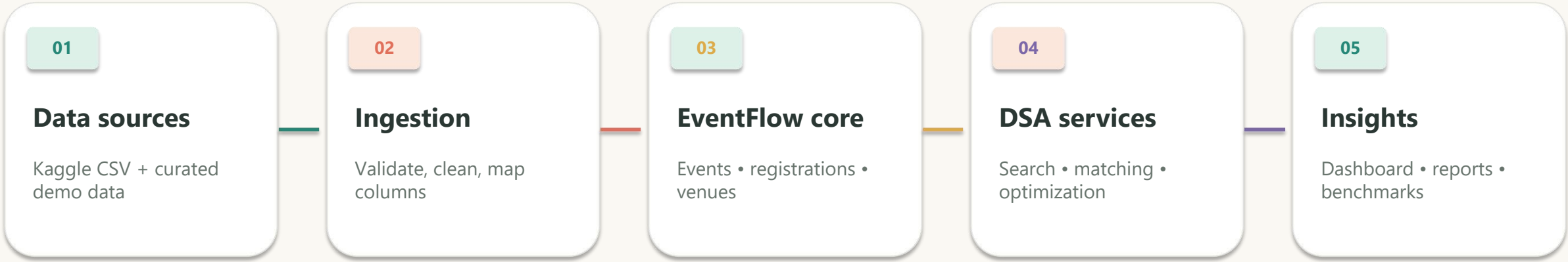
## Challenges in event operations

- Finding events across long descriptions
- Detecting similar or repeated text
- Assigning people to eligible activities
- Managing venue capacity and schedule conflicts
- Understanding growing registration data

## Project objectives

- Centralize event and registration records
- Demonstrate fast search and text indexing
- Model allocation with graph algorithms
- Explore scheduling constraints and approximations
- Generate useful summaries from imported data

# Platform scope & system flow




**Design principle: imported source records remain distinguishable from synthetic supporting records created for missing fields.**

# Six DSA modules, one practical platform

M1

## String algorithms

KMP • Z • Rabin–Karp • Aho–Corasick

M2

## Suffix structures

Suffix array • SA-IS • LCP/Kasai • automaton

M3

## Dynamic programming

Edit distance • bitmask DP • matrix chain • OBST

M4

## Network flow

Ford–Fulkerson • Edmonds–Karp • Dinic • matching

M5

## NP-completeness

SAT/3-SAT • reductions • Vertex Cover approximation

M6

## Randomized & parallel

QuickSort • reservoir sampling • scan • reduce

# M1 + M4

M1

## String algorithms

Use KMP, Z-function, Rabin–Karp and Aho–Corasick for event names, booking codes and multi-keyword document search.

### Practical output

Matching positions, highlighted keywords and search comparisons.

M4

## Network flow & matching

Model venue capacity as flow networks and assign eligible staff or volunteers to activities with bipartite matching.

### Practical output

Capacity analysis, maximum flow, assignments and conflict checks.

**Deliverable focus: correct algorithm implementations, test cases, complexity notes and a clear visual explanation.**

# M2 + M5 |

M2

## Suffix structures

Build suffix-array indexes, LCP/Kasai analysis and substring search to discover repeated phrases across event documents.

### Practical output

Indexed text, common-substring results and similarity insights.

M5

## Constraints & approximation

Represent schedule constraints with SAT/3-SAT, transform graphs and demonstrate a 2-approximation for Vertex Cover.

### Practical output

Constraint graphs, reduction walkthroughs and covered-conflict views.

**Deliverable focus: explain the logic behind each structure or reduction and verify it with small test instances.**

# M3 + M6 |

M3

## Advanced dynamic programming

Use edit distances for spelling correction, bitmask DP for small activity sets, matrix-chain optimization and Optimal BST demonstrations.

### Practical output

Correction suggestions, selection results and DP tables.

M6

## Randomized & parallel algorithms

Demonstrate randomized QuickSort, reservoir sampling, Miller–Rabin, Blelloch Scan, parallel reduction and Brent's theorem.

### Practical output

Sorted rankings, stream samples, prefix totals and benchmark comparisons.

**Deliverable focus: show correct outputs, edge cases, complexity and honest benchmark measurements.**

# Dataset strategy: Kaggle + transparent enrichment

### 01 Source

Select a relevant Kaggle event, registration, attendance or conference dataset. Record the source URL and license.

### 02 Prepare

Inspect columns, clean dates, normalize categories, report missing values and deduplicate records.

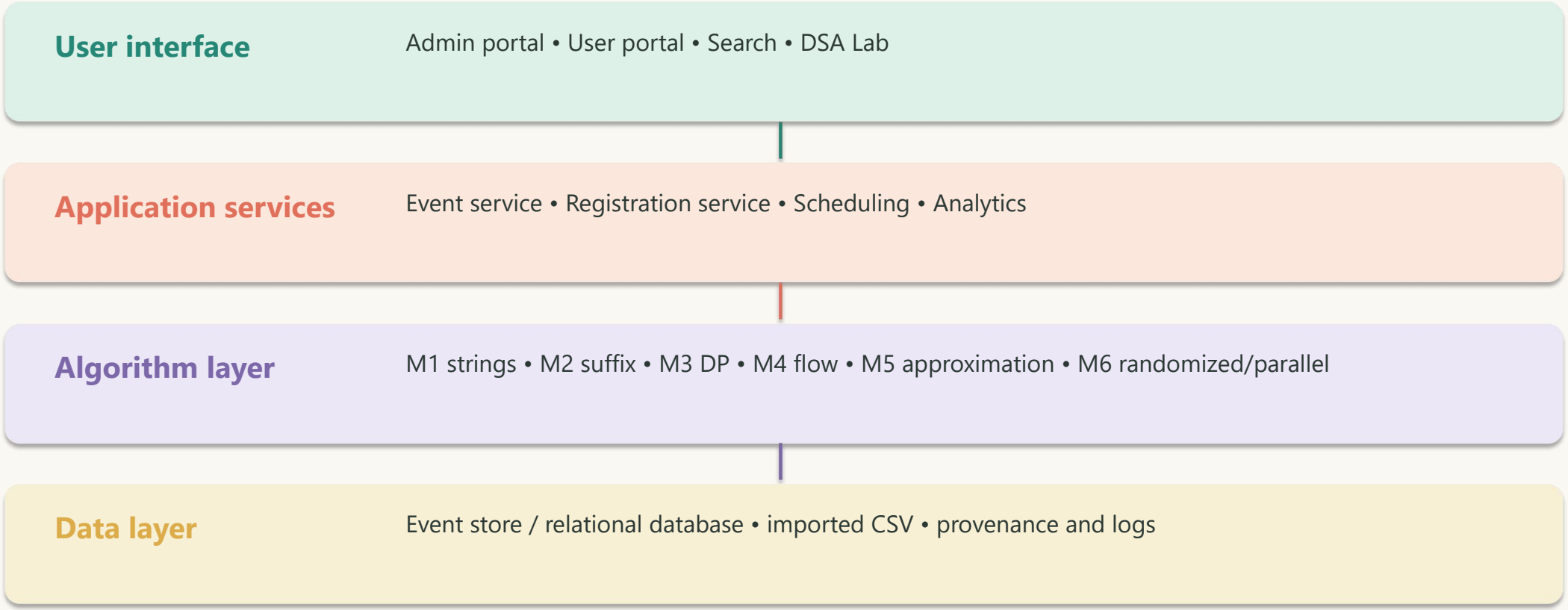
### 03 Enrich carefully

Create synthetic venue, staff or schedule records only where source data is missing—and label them clearly.

**Data integrity rule: never describe synthetic records as Kaggle data, and never claim a dataset was downloaded until it is verified.**

Recommended evidence in final report: dataset name • source link • fields used • row count • cleaning summary

# Proposed technical architecture




*Current uploaded prototype: Java console application with Admin/User portals, event storage and several M1/M2 search features.*

# Prototype baseline & development roadmap

## Visible in the provided codebase

- Java application entry point with Admin and User menus
- Add, view and delete event workflows
- Event search engine with Naive, KMP, Z &Rabin–Karp implementations
- Aho–Corasick keyword detection
- Suffix-array-related CO2 class and edit-distance support
- Local events.txt data file

## Next implementation / verification

- Verify and document Kaggle dataset import
- Review correctness and tests for every algorithm
- Complete M3–M6 modules and demonstrations as needed
- Add venue, schedule and staff allocation workflows
- Build analytics, UI polish and repeatable test instructions
- Record actual test results before final submission

*Status note: this deck distinguishes features visible in the uploaded Java prototype from the broader target scope.*

# Suggested live project demonstration

| | | |
|---|---|---|
| 1 | **Open EventFlow** | Show the Admin/User entry menu and event list. |
| 2 | **Create an event** | Add a sample event and show its stored fields. |
| 3 | **Search intelligently** | Compare search methods and run keyword detection. |
| 4 | **Explore text similarity** | Demonstrate suffix/LCP or edit-distance outputs available in the build. |
| 5 | **Show DSA modules** | Walk through the M1–M6 demos, clearly noting completed vs planned modules. |
| 6 | **Explain data provenance** | Show Kaggle source details, import summary and synthetic-data labels. |

# Expected outcomes & responsible claims

## Expected outcomes

- Easier event discovery and record management
- Demonstrable search and text-indexing techniques
- Explainable resource-allocation models
- Visual learning environment for DSA concepts
- Reproducible dataset preparation and tests

## What makes the project credible

- Real source data with traceable provenance
- Synthetic data clearly labelled
- Algorithm outputs checked against tests
- Performance claims backed by measurements
- Clear separation of implemented and planned features

**EventFlow turns core DSA concepts into practical, explainable event-management workflows.**

THANK YOU

# EventFlow

**Smarter events. Better coordination. Explainable algorithms.**